ТЕХНИЧЕСКОЕ ЗАДАНИЕ И СТАРТОВЫЙ ПРОМПТ • ВЕРСИЯ 1.0

БОТ-БУХГАЛТЕР (VIRTUAL ARBITRAGE ENGINE)

Архитектура виртуального арбитража с балансировкой 50/50, двухфазным циклом сделки, учётом комиссий в PostgreSQL и отчётностью в Telegram

Базовый репозиторий	`crypto_screener` (общий конфиг, `AsyncHttpClient`, `DatabaseManager`)
Биржи в контуре (5 шт.)	Bybit, OKX, KuCoin, Gate.io, BingX (спотовый рынок, виртуальные счета)
База данных и стек	PostgreSQL (`crypto_db`), Python 3.11+, `asyncio`, `aiohttp`, Telegram Bot API
Режим работы	Симуляция на виртуальных балансах без риска потери реального капитала

1. Подробная архитектура «Бота-бухгалтера»

1.1. Источник сигналов и отказ от публичного сканирования

Бот-бухгалтер работает как исполнительное и учётное ядро поверх уже работающего скринера. Он не тратит лимиты и сетевые ресурсы на опрос сотен пар на биржах — все потенциальные связки он забирает напрямую из таблицы `possible_trades` в PostgreSQL (`crypto_db`).

- **Отсутствие приватных ключей на старте:** На этапе обкатки логики и сбора статистики реальные API-ключи бирж с правами на торговлю и вывод средств не используются.
- **Проверка жизнеспособности (Pre-Trade Check):** Получив свежий сигнал из БД, бот не исполняет его слепо. Он сверяет актуальность сигнала, проверяет достаточность виртуальных остатков на обеих биржах и убеждается, что спред с учётом всех комиссий (торговых и будущей ребалансировки) даёт положительное математическое ожидание.

1.2. Модель распределения виртуального депозита (50% / 50%)

По итогам прогона скринера отбирается фиксированный пул ликвидных и стабильных монет (например, 5–10 пар к USDT). Весь виртуальный стартовый капитал делится в равных долях:

- **Равномерное распределение по биржам и парам:** Общий банк делится поровну между всеми 5 биржами (`Bybit`, `OKX`, `KuCoin`, `Gate.io`, `BingX`) и далее поровну между выбранными торговыми парами.
- **Структура кошелька 50% COIN / 50% USDT:** На каждой бирже по каждой рабочей монете половина выделенной суммы хранится в самой монете (`COIN`), а вторая половина — в стейблкоине (`USDT`).
- **Зачем это нужно:** Такая структура позволяет при появлении арбитражного окна мгновенно (в одну секунду) купить монету за `USDT` на дешёвой бирже и одновременно продать имеющуюся монету за `USDT` на дорогой бирже — без ожидания межбиржевых блокчейн-переводов и риска проскальзывания цены.

1.3. Двухфазный жизненный цикл арбитражной сделки

Каждая арбитражная операция разбита на два независимых по времени этапа:

- **Фаза 1 — Мгновенный вход в арбитраж (Хеджированное исполнение):**
  - Срабатывает сигнал из `possible_trades` (например, `Биржа А` дешевле, `Биржа Б` дороже).
  - Бот проверяет: на `Бирже А` есть свободный `USDT`, на `Бирже Б` есть свободный `COIN`, пара не заблокирована в другой незавершённой сделке.
  - Одновременно фиксируются два виртуальных ордера: покупка объёма `Q` на `Бирже А` и продажа того же объёма `Q` на `Бирже Б`.
  - С балансов виртуально списываются торговые комиссии (`taker/maker fee` из справочника БД). После Фазы 1 прибыль по сделке уже зафиксирована в разнице `USDT`, но возникает перекос: на `Бирже А` стало много монет, а на `Бирже Б` — много `USDT`.
- **Фаза 2 — Спокойная ребалансировка и закрытие цикла:**
  - Сделка переходит в статус `PENDING_REBALANCE` (ожидает выравнивания). Никакой спешки нет, так как рыночный риск уже закрыт.
  - Бот в фоновом режиме анализирует самый дешёвый способ вернуть пропорцию 50/50 на обеих биржах:
    1. *Обратный спред (Zero-Fee Rebalance):* Если цена на рынке качнулась в обратную сторону, бот совершает обратную сделку (продаёт лишний `COIN` на `Бирже А` и покупает на `Бирже Б`), зарабатывая второй раз и экономя на сети.
    2. *Виртуальный сетевой перевод:* Если обратного спреда долго нет, бот рассчитывает перегон монет с `Биржи А` на `Биржу Б` через самую дешёвую сеть из справочника комиссий БД (например, BEP20 / TRC20 / Arbitrum / TON) и перегоняет эквивалент `USDT` обратно (или через дешёвую транзитную монету).
  - После выравнивания балансов из валовой прибыли вычитаются все фактические затраты на ребалансировку, и цикл закрывается в статус `CLOSED` с записью чистого P&L.

1.4. Учёт затрат, статистика и Telegram-отчётность

- **Единая база комиссий и затрат:** В PostgreSQL хранятся актуальные торговые комиссии бирж и комиссии на вывод/перевод по сетям, чтобы ни один виртуальный цент не терялся.
- **Глубокая статистика:** Фиксируется каждый шаг — время входа, время ожидания ребалансировки, все затраты, чистый профит в USDT и % годовых/суточных на вложенный депозит.
- **Интеграция с Telegram:** Все уведомления отправляются асинхронно через общий `AsyncHttpClient` (через Xray-туннель на VPS). Бот присылает алерты об открытии Фазы 1, закрытии Фазы 2 и регулярные сводки по балансам и P&L.

2. Готовый промпт для старта новой сессии с ИИ

Скопируй текст из блока ниже и прикрепи этот PDF (вместе с паспортом `crypto_screener`), когда будешь готов начать писать код:

ПРОМПТ-ШАБЛОН ДЛЯ ЗАПУСКА РАЗРАБОТКИ «БОТА-БУХГАЛТЕРА»

Привет, бро! Продолжаем работу над проектом `crypto_screener`. Всю базу по скринеру и подробное ТЗ на «Бота-бухгалтера» я прикрепил в PDF.

Начинаем разработку «Бота-бухгалтера» поверх существующей архитектуры проекта (`config/config.py` , `AsyncHttpClient` из `network/client.py` , `DatabaseManager` из `database/db_manager.py`).

Ключевая логика на первый этап:
1. Никаких реальных API-ключей бирж и реальных торгов — работаем строго на виртуальных балансах в PostgreSQL (`crypto_db`).
2. Стартовый виртуальный банк делим поровну на 5 бирж (Bybit, OKX, KuCoin, Gate.io, BingX) и выбранные торговые пары в пропорции 50% COIN / 50% USDT на каждой бирже.
3. Сигналы забираем из таблицы `possible_trades` . Перед входом проверяем жизнеспособность связки и достаточность виртуальных остатков.
4. Реализуем двухфазный цикл сделки: Фаза 1 (одновременная виртуальная покупка и продажа на двух биржах с вычетом торговых комиссий) и Фаза 2 (спокойный поиск самого выгодного способа выровнять балансы обратно к 50/50 — через обратный спред или виртуальный перевод с учётом сетевых комиссий из БД — и фиксация чистой прибыли).
5. Все комиссии, ордера, сделки и статистику храним в PostgreSQL, а отчёты и периодические сводки отправляем в Telegram через `AsyncHttpClient` .

Наши правила разработки остаются прежними: нулевой хардкод (всё в `config/config.py` с явной ошибкой при отсутствии ключа), отступы табуляцией, подробные комментарии на русском языке и «рентген-тесты» на моках перед деплоем на сервер.

Давай начнём с Шага 1: спроектируем SQL-схему новых таблиц в PostgreSQL и расширим `DatabaseManager` .

3. План действий и строгая очерёдность разработки

Чтобы система получилась надёжной, без костылей и с прозрачной математикой, двигаемся строго по следующим 6 этапам:

Этап	Модули и файлы	Что делаем и какой результат получаем
Шаг 1 **Фундамент БД**	`database/db_manager.py` (таблицы в `crypto_db`)	Проектируем и создаём 4 таблицы: `virtual_balances` (остатки COIN и USDT по биржам), `exchange_fees` (справочник торговых и сетевых комиссий), `virtual_orders` (каждый виртуальный ордер) и `arbitrage_deals` (связка Фазы 1 и Фазы 2 с расчётом чистого P&L). Добавляем методы транзакционного обновления балансов.
Шаг 2 **Конфиг и банк**	`config/config.py` `accountant/init_bank.py`	Добавляем секцию `CONFIG["accountant"]` (общий банк в USDT, список отобранных пар, пороги ребалансировки, интервал отчётов, токен и `chat_id` Telegram). Пишем скрипт первичной инициализации виртуальных счетов 50/50 и заполнения базовых комиссий.
Шаг 3 **Telegram-модуль**	`notifications/telegram.py`	Создаём асинхронный клиент уведомлений поверх `AsyncHttpClient` (чтобы запросы к `api.telegram.org` на сервере шли через рабочий Xray-туннель). Настраиваем форматированные шаблоны: вход в сделку, закрытие ребалансировки, периодическая сводка балансов и P&L.
Шаг 4 **Ядро Фазы 1**	`accountant/executor.py`	Реализуем чтение новых сигналов из `possible_trades`, проверку «жизнеспособности» пары (фильтр дублей, проверка остатков 50/50, расчёт чистого профита после всех комиссий) и атомарное виртуальное исполнение одновременной покупки и продажи.
Шаг 5 **Ядро Фазы 2**	`accountant/rebalancer.py`	Реализуем фоновый модуль выравнивания перекосов: проверка появления обратного спреда для бесплатной ребалансировки или расчёт оптимального маршрута виртуального перевода по минимальной комиссии сети из `exchange_fees`, закрытие сделки и фиксация итогового P&L.
Шаг 6 **Рентген-тесты и запуск**	`tests/test_accountant_*.py` `run_accountant.py`	Пишем изолированные тесты на моках (проверка математики 50/50, двойных комиссий, защиты от ухода баланса в минус, сценариев Фазы 1 и Фазы 2). Запускаем бота на VPS в отдельной `screen`-сессии рядом со скринером.

Важное правило перед стартом: Сначала завершаем 24-часовой прогон скринера `crypto_screener` на боевом сервере, выгружаем из `possible_trades` статистику по самым частым и стабильным связкам, утверждаем список из 5–10 монет — и только после этого запускаем инициализацию виртуального банка из Шага 2.